

Atividade Prática: Containerização de API

Metodologia: Sala de Aula Invertida

1. Objetivo

Dando continuidade à construção da **API de Estoque** abordada no último laboratório, o próximo passo na evolução da nossa arquitetura é o empacotamento da aplicação. Vocês deverão substituir o script imperativo de instalação por uma abordagem moderna e declarativa utilizando **Docker**.

2. Etapa de Pesquisa (Entregável)

Antes de implementar o container da API de Estoque, é fundamental compreender a semântica da construção de imagens. Pesquisem na documentação oficial do Docker e redijam um **documento curto (PDF, Markdown ou TXT)** explicando detalhadamente com suas próprias palavras para que servem as seguintes instruções de um Dockerfile:

FROM

WORKDIR

COPY

RUN

EXPOSE

CMD

3. Missão: Mão na Massa

Com a pesquisa concluída, abra o diretório da `estoque-api`. Crie um arquivo `Dockerfile` na raiz do projeto com o objetivo de isolar e rodar a nossa aplicação Node.js. O container de vocês deve obrigatoriamente atender aos seguintes critérios:

- Partir de uma imagem base oficial adequada para executar projetos em Node.js (procurem por versões otimizadas/leves).
- Copiar todos os arquivos essenciais da API para dentro do container, garantindo que o código-fonte esteja acessível.
- Executar a instalação de todas as dependências mapeadas no gerenciador de pacotes do Node.
- Expor a mesma porta utilizada na nossa configuração original do servidor (Porta 3000).
- Definir a instrução de inicialização padrão para que a API comece a rodar assim que o container for instanciado.

Validação Prática:

Descubram e utilizem os comandos de terminal necessários para fazer o **build** da imagem localmente e executar (**run**) o container. O desafio é mapear a porta corretamente para que vocês consigam acessar a rota de status da API no navegador da máquina de vocês.

4. Material de Referência (Exemplo em Python)

Para auxiliar na visualização, o bloco abaixo demonstra a estrutura de um `Dockerfile` utilizado em um ambiente distinto: uma aplicação web escrita em **Python** utilizando o framework Flask. Analisem como o fluxo lógico é construído.

Atenção: Vocês não podem copiar este código. Ele serve apenas como um mapa mental. Vocês precisarão traduzir essas etapas e ferramentas para o ecossistema e as necessidades da nossa API em Node.js.

```
# 1. Definição da imagem base com Python 3.10
FROM python:3.10-slim

# 2. Definição do diretório de trabalho interno
WORKDIR /app

# 3. Cópia estratégica do arquivo de dependências (Cache)
COPY requirements.txt .

# 4. Execução da instalação das bibliotecas Python
RUN pip install --no-cache-dir -r requirements.txt

# 5. Cópia do restante do código-fonte para o container
COPY . .

# 6. Documentação da porta utilizada pelo Flask
EXPOSE 5000

# 7. Comando de inicialização do servidor
CMD ["python", "app.py"]
```